NexSpace

Backend & Agent Controller PRD Technical Audit

Comprehensive Codebase Inspection, Ground-Truth Verification & Gap Analysis

---

EXECUTIVE AUDIT SUMMARY:

This technical audit evaluates the actual implemented backend code in ml_backend/, processing/, server/services, and Next.js API routes against the NexSpace PRD. Each capability is classified across five ground-truth states: REAL & WORKING, PARTIALLY IMPLEMENTED, MOCK/SIMULATED, MISSING, or BROKEN.

EXECUTIVE AUDIT ANSWERS TO KEY QUESTIONS

- **Which specialist models are actually loaded & used?:** None loaded by default. tools.py wraps google/paligemma-3b-ft-rsvqa-lr-224 and Salesforce/blip-image-captioning-base in deferred try...except blocks; if weights are absent, it returns deterministic string fallbacks.
- **Are PaliGemma / BLIP / YOLO real inference or mock logic?:** PaliGemma and BLIP have real Hugging Face pipeline scaffolding but default to mock strings. YOLOv8 is 100% simulated via detect_objects_placeholder() returning hardcoded vessel/infrastructure coordinates.
- **Is Agent Controller agentic or rule-based?:** Rule-based regex & heuristic keyword matching in router.py (e.g. re.search(r'\bhow many\b')). No autonomous LLM function-calling graph is active.
- **Does Optical + SAR fusion actually exist?:** No joint tensor embedding fusion exists. In orchestrator.py, fusion is purely string concatenation of optical_caption_result and sar_caption_result.
- **Are execution traces real or simulated?:** Simulated. Stage graphs and timing in queryService.ts and /api/query/[id]/trace return static mock timestamps and hardcoded GPU allocations.
- **Are confidence scores real or hardcoded?:** Hardcoded. Sets confidence to 0.32 if 'how many' is present, and 0.85 for all other queries.
- **Are evidence bounding boxes from models or hardcoded?:** Hardcoded static coordinate arrays in ComparisonPage, ScanResultsPage, and pipeline.py.
- **Does Next.js /api/query communicate with FastAPI backend?:** Yes, attempts fetch('http://localhost:8000/api/query') with a 3s timeout and falls back to an inlined TypeScript rule router if FastAPI is offline.

Audit Timestamp: August 2026 · Local Workspace Distribution · NexSpace Platform v2.4.1

1. Agent Controller Implementation Audit

Detailed capability evaluation against PRD specifications

Requirement:

Current Component: ml_backend/router.py (_is_counting, _is_open_ended), app/api/query/route.ts
Current Status: **PARTIALLY IMPLEMENTED (Regex & Keyword Heuristics)**
What it currently does: Uses regex patterns and starter phrase arrays (e.g., 'what is visible', 'is there') to classify queries into counting, open-ended, or closed-ended categories.
What the PRD requires: Autonomous semantic intent classification capable of parsing complex geospatial constraints (bounding boxes, sensor modalities, temporal windows, and confidence thresholds).
Exact change needed: Replace or augment keyword heuristics with an LLM function-calling schema or small classification model.
Dependencies / Blockers: LLM API Key (Anthropic/Gemini/OpenAI) or local quantized intent classifier model.

Requirement:

Current Component: ml_backend/router.py (_normalize_closed_ended, _normalize_counting, decompose_open_ended)
Current Status: **REAL AND WORKING (Heuristic Engine)**
What it currently does: Decomposes open-ended queries into atomic RSVQA binary sub-questions using static probe dictionaries (FEATURE_PROBES) and formats questions with trailing '?' and leading capitals.
What the PRD requires: Decompose complex geospatial queries into atomic, verifiable sub-queries for visual verification against multi-spectral satellite tiles.
Exact change needed: Expand static probe dictionary to dynamic zero-shot spatial entity extraction.
Dependencies / Blockers: None.

Requirement:

Current Component: ml_backend/router.py, ml_backend/orchestrator.py
Current Status: **PARTIALLY IMPLEMENTED (Captioning only, Grounding missing)**
What it currently does: Routes open-ended requests to Optical_Caption or SAR_Caption and invokes CaptioningTool.caption().
What the PRD requires: Free-form captioning coupled with visual grounding (returning localized bounding boxes and segmentation masks for referenced scene objects).
Exact change needed: Integrate visual grounding model (Grounding-DINO or PaliGemma grounding tokens <loc0000>...<loc1023>).
Dependencies / Blockers: Grounding model weights and PyTorch GPU runtime.

Requirement:

Current Component: ml_backend/router.py (line 172), ml_backend/change_analysis.py
Current Status: **REAL (Classical Diff) / MISSING (Change-VQA)**
What it currently does: Triggers pixel differencing pipeline when both change_image_a and change_image_b are supplied.
What the PRD requires: Bi-temporal pixel diff heatmap generation combined with Change-VQA (answering natural language questions about what changed between T0 and T1).
Exact change needed: Feed bi-temporal image pair and difference mask into a multimodal VLM to answer natural language questions about detected changes.
Dependencies / Blockers: Change-VLM model or dual-image prompt pipeline.

Requirement:

Current Component: ml_backend/router.py (lines 190, 213), ml_backend/orchestrator.py
Current Status: **PARTIALLY IMPLEMENTED (Dispatches separate text captions)**
What it currently does: Detects boolean flags has_optical and has_sar and dispatches separate calls to Optical_Caption and SAR_Caption.
What the PRD requires: Co-registered dual-modal fusion that analyzes complementary strengths (optical multi-spectral reflectance + SAR backscatter/dielectric roughness).
Exact change needed: Implement dual-channel ingestion and cross-modality reasoning pipeline.
Dependencies / Blockers: Paired co-registered optical/SAR sample datasets.

---

1. Agent Controller Audit (Continued)

Validation, Tool Execution, Output Fusion & Traces

Requirement:
Current Component: ml_backend/server.py (decode_b64_image), app/api/query/route.ts
Current Status: **PARTIALLY IMPLEMENTED (Format decode only)**
What it currently does: Validates base64 string decoding and handles empty query strings.
What the PRD requires: Validation of raster format (GeoTIFF/JP2), spatial resolution/GSD, Coordinate Reference System (CRS), channel count, and coregistration alignment.
Exact change needed: Add metadata validation schema (CRS check via rasterio.crs, spatial bounds matching, band validation).
Dependencies / Blockers: rasterio / GDAL in ml_backend.

Requirement:
Current Component: ml_backend/router.py (route)
Current Status: **REAL (Deterministic Rules)**
What it currently does: Populates target_tools list (VQA, Optical_Caption, SAR_Caption, Change_Analysis) based on intent heuristics.
What the PRD requires: Dynamic tool invocation with specialized parameter configurations (e.g. confidence thresholds, change sensitivity, target categories).
Exact change needed: Pass runtime parameters (change_threshold, iou_threshold, spectral_indices) from the query down to tool invocations.
Dependencies / Blockers: Query frontend passing filter state in payload.

Requirement:
Current Component: ml_backend/orchestrator.py (lines 64-78)
Current Status: **PARTIALLY IMPLEMENTED (Synchronous execution)**
What it currently does: Iterates through target_tools synchronously and invokes tool instances.
What the PRD requires: Async concurrent tool execution with timeout handling, GPU batch scheduling, and error isolation.
Exact change needed: Wrap tool calls in asyncio.gather() with individual exception handling so one tool failure does not abort the entire pipeline.
Dependencies / Blockers: None.

Requirement:
Current Component: ml_backend/orchestrator.py (_synthesize)
Current Status: **REAL (Markdown Template Synthesis)**
What it currently does: Combines optical description, SAR description, structured VQA bullet points, and change summary into a Markdown document.
What the PRD requires: Unified synthesis resolving conflicts between tools and presenting structured intelligence findings.
Exact change needed: Add conflict resolution logic (e.g., if optical caption says clear water but SAR shows high metallic backscatter, surface an explicit vessel anomaly note).
Dependencies / Blockers: None.

Requirement:
Current Component: ml_backend/router.py, ml_backend/tools.py (VQAResult.low_confidence)
Current Status: **PARTIALLY IMPLEMENTED (Hardcoded threshold logic)**
What it currently does: Flags queries with 'how many' with requires_count_warning = True and hardcodes confidence to 0.32 vs 0.85.
What the PRD requires: Extraction of true model softmax probabilities, temperature scaling, and calibrated uncertainty intervals.
Exact change needed: Extract model output token logits/probabilities from Hugging Face pipeline outputs and compute true confidence scores.
Dependencies / Blockers: Model execution with logits output enabled.

Requirement:
Current Component: server/services/queryService.ts (generateExecutionStages), app/api/query/[id]/trace/route.ts
Current Status: **MOCK / SIMULATED**
What it currently does: Generates static simulated stages (Data Ingestion, Radiometric Correction, Neural Feature Extraction, etc.) with mock progress and timestamps.
What the PRD requires: Real execution telemetry capturing actual execution stage start times, duration in ms, GPU memory allocations, and tensor shape metadata.
Exact change needed: Instrument orchestrator.py and Next.js pipeline stages with real execution timers and telemetry log capture.
Dependencies / Blockers: None.

---

2. Specialist Tools Audit

VQA, Captioning, Change Analysis & Fusion Engines

Requirement:
Current Component: ml_backend/tools.py (VQATool)
Current Status: **PARTIALLY IMPLEMENTED / FALLBACK MOCK**
What it currently does: Configured to load google/paligemma-3b-ft-rsvqa-lr-224 via Hugging Face pipeline('image-text-to-text'). If model load fails or weights are absent, returns deterministic mock answers ('yes', '12 (estimated)').
What the PRD requires: Real tensor execution on optical/multispectral imagery answering closed-ended spatial presence, attribute, and count queries.
Exact change needed: Download and cache model weights (or configure an API inference endpoint / ONNX runtime) and execute real inference.
Dependencies / Blockers: Hugging Face token (model is gated) and GPU/CPU RAM (~6GB VRAM for 3B FP16).

Requirement:
Current Component: ml_backend/tools.py (CaptioningTool)
Current Status: **PARTIALLY IMPLEMENTED / FALLBACK MOCK**
What it currently does: Configured for Salesforce/blip-image-captioning-base. Returns mock scene descriptions when weights are not downloaded locally.
What the PRD requires: Automated natural language description of satellite tile land cover, infrastructure, and maritime activity.
Exact change needed: Load BLIP / BLIP-2 checkpoint and run pipeline(image).
Dependencies / Blockers: transformers, torch, pillow installed in environment.

Requirement:
Current Component: ml_backend/change_analysis.py
Current Status: **REAL AND WORKING (Classical Diff Engine)**
What it currently does: Converts image pair to grayscale numpy arrays, calculates pixel intensity diff |B - A|, normalizes dynamic range, builds RGBA heatmap, alpha-blends overlay, calculates changed pixel fraction, and produces human-readable text summary.
What the PRD requires: Co-registered bi-temporal change detection with pixel heatmap, delta percentage, and anomaly categorization.
Exact change needed: Working as designed for classical diff. Needs connected anomaly clustering (contour extraction) to output labeled bounding boxes dynamically.
Dependencies / Blockers: None.

Requirement:
Current Component: ml_backend/orchestrator.py
Current Status: **MISSING (Only text concatenation exists)**
What it currently does: Calls optical tool and SAR tool independently and lists both text captions in markdown.
What the PRD requires: Joint feature extraction or multi-sensor fusion model leveraging SAR backscatter penetration and optical spectral reflection.
Exact change needed: Implement dual-channel feature merging or a multimodal prompt pipeline passing both sensor representations.
Dependencies / Blockers: Paired Optical/SAR training or inference model.

---

3. Input / Ingestion & Pipeline Audit

Requirement:
Current Component: ml_backend/server.py, server/services/ingestionService.ts
Current Status: **PARTIALLY IMPLEMENTED (Format check only)**
What it currently does: Verifies file extension (.tif, .jp2, .zip) in Next.js and base64 decode validity in FastAPI.
What the PRD requires: Automated band detection (RGB, NIR, SWIR, SAR VV/VH), bit-depth normalization (uint8, uint16, float32), and modality tagging.
Exact change needed: Inspect image channel headers via PIL.Image or rasterio and tag modality automatically.
Dependencies / Blockers: None.

Requirement:
Current Component: processing/pipeline.py, app/upload/page.tsx
Current Status: **PARTIALLY IMPLEMENTED / MOCK**
What it currently does: Next.js upload checklist displays simulated CRS (EPSG:32651) and GSD (10m). processing/pipeline.py computes real NDVI math ((NIR - Red) / (NIR + Red)) but does not parse geospatial geotags.
What the PRD requires: Extraction of real GeoTIFF affine transform matrices, ground sample distance (GSD), and re-projection to UTM / EPSG:4326.
Exact change needed: Utilize rasterio.open() in processing/main.py to extract real CRS and resolution metadata.
Dependencies / Blockers: rasterio package in processing service.

Requirement:
Current Component: ml_backend/change_analysis.py (lines 62-67), processing/pipeline.py
Current Status: **PARTIALLY IMPLEMENTED**
What it currently does: Checks image_a.size == image_b.size and resizes B to match A if dimensions differ.
What the PRD requires: Feature matching (e.g. ORB / SIFT + RANSAC homography) to compute real co-registration RMS error and warp images prior to differencing.
Exact change needed: Implement OpenCV ORB feature alignment in change_analysis.py to calculate true coregistration RMS pixel error.
Dependencies / Blockers: opencv-python-headless.

---

4. Executive Classification & Recommended Roadmap

Categorized status and implementation order

1. Already Working

• Classical Pixel-Level Change Analysis (ml_backend/change_analysis.py): Fully functional numpy differencing, dynamic range normalization, RGBA heatmap creation, and alpha overlay generation.

• Deterministic Query Structuring & Decomposition (ml_backend/router.py): Rule-based normalization of closed-ended and counting queries into RSVQA format with structured sub-questions.

• Low-Confidence Warning Flagging (ml_backend/router.py): Automatically flags queries with 'how many' and injects estimation warnings into synthesized output.

• FastAPI Query & Change Analysis API (ml_backend/server.py): Functional REST API endpoints (/api/query, /api/change-analysis, /api/health) with base64 image decoding/encoding.

• Next.js API Fallback Layer (app/api/query/route.ts): Reliably calls FastAPI backend with a 3-second timeout and fails gracefully to embedded router logic when backend is offline.

• Spectral NDVI Computation (processing/pipeline.py): Real mathematical implementation of normalized difference vegetation index on multi-band numpy arrays.

2. Needs Modification

• ml_backend/tools.py: Download/cache real model checkpoints or connect an active inference runtime so real VQA and Captioning inference execute instead of falling back to default strings.

• ml_backend/change_analysis.py: Add OpenCV ORB feature alignment for true co-registration validation and extract bounding boxes from changed connected components.

• app/api/query/route.ts & app/components/QueryPage.tsx: Connect frontend advanced filter controls (date range, sensor modality, resolution, cloud cover) so they are passed in the request body to the backend.

• server/services/queryService.ts: Replace hardcoded execution stages with real timing and telemetry instrumentation.

3. Missing

• Change-VQA: Multi-modal reasoning over bi-temporal pairs to answer textual questions about what changed.

• Optical + SAR Feature Fusion: Joint multi-sensor representation model (currently only text output concatenation exists).

• True Model Confidence Calculation: Extracting real token logits/probabilities from model outputs rather than using hardcoded values (0.32 / 0.85).

• Real Object Detection / Grounding: YOLOv8 or Grounding-DINO inference (currently simulated via detect_objects_placeholder).

• GeoTIFF / CRS Metadata Extraction: Parsing affine transforms and EPSG projection headers from GeoTIFF files via rasterio.

4. Blocked by Another Team / Component

• Real DB Persistence: Prisma ORM schema is complete in prisma/schema.prisma, but blocked by missing DATABASE_URL / .env configuration and inactive PostgreSQL instance.

• Live Model Inference: Blocked by missing Hugging Face authentication token for the gated PaliGemma model (google/paligemma-3b-ft-rsvqa-lr-224) and local GPU/VRAM setup.

• Unified Full-Stack Startup: Next.js (npm run dev) and FastAPI (uvicorn server:app) must be run concurrently.

5. Recommended Implementation Order

1. Step 1: Environment & Full-Stack Startup: Configure .env.local and add a unified dev script (npm run dev starting both Next.js and FastAPI) so communication is live.

2. Step 2: Frontend Filter Connection: Update QueryPage.tsx to pass selected filter state to /api/query and router.py.

3. Step 3: Specialist Tool Weights & Model Runtime: Configure model downloading/caching in tools.py with real logit confidence scoring.

4. Step 4: Dynamic Anomaly Extraction & Alignment: Add OpenCV co-registration and contour detection in change_analysis.py to output real anomaly bounding boxes.

5. Step 5: Real Execution Telemetry: Instrument orchestrator.py to record real stage durations and pipe them into the execution trace views.

6. Step 6: Database Integration: Initialize PostgreSQL container via docker-compose.yml and run npx prisma db push to enable persistence.